SMU Beamer Theme

Complete Feature Demonstration

Your Name

School of Computing and Information Systems
Singapore Management University

January 26, 2026

Overview

- Basic Settings
  - Introduction
  - Text Formatting
- Components Design
  - Block Environments
  - Column Layouts
  - Tables and Lists
- Code Rendering
  - Code Highlighting
  - Summary

SMU
SINGAPORE MANAGEMENT UNIVERSITY

01 Basic Settings

About This Theme

The SMU Beamer Theme provides a LaTeX beamer template based on SMU's official PowerPoint template, with enhanced features for academic presentations:

- Official SMU branding and colors
- Code highlighting with minted
- Custom block components
- Flexible layout options

Key Features

Design & Layout:

- Professional SMU branding
- Progress bar footer
- Section transition pages
- Multi-column layouts

Text & Colors:

- Custom color commands
- Text highlighting
- Bold color variants

Code Highlighting:

- Minted integration
- 10+ programming languages
- 49 built-in themes
- Inline code support

Custom Blocks:

- Standard blocks
- Alert blocks
- Mathematical boxes (Theorem, etc.)
- Tagged blocks

Basic Text Formatting

Font Styles:

- Regular text in Calibri font
- **Bold text appears in SMU blue**
- *Italic text for emphasis*
- **Important warning** using red bold
- **Key concept** using blue bold

Inline Code Style:

- Use `inline code` for technical terms
- Function names like `printf()` or `main()`
- Styles can be customized in `smucore/smucode.def`

Custom Colors

SMU Brand Colors:

- SMU Blue - Primary color
- SMU Light Blue - Secondary color
- SMU Gold - Accent color
- SMU Green - Additional accent

Additional Accent Colors:

- SCIS Yellow - School of Computing
- Alert Red - For warnings
- Muted Blue - Alternative blue
- Muted Green - Alternative green

Mathematical Formulas

Inline Mathematics:

Einstein's famous equation $E = mc^2$, Euler's number $e \approx 2.71828$, and pi $\pi \approx 3.14159$.

Display Equations:

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}$$

Aligned Equations:

$$\begin{aligned} f(x) &= x^2 + 2x + 1 \\ &= (x+1)^2 \\ &= (x+1)(x+1) \end{aligned}$$

Text & Mathematical Highlighting

Basic Highlighting:

Normal text with default highlighted content using SMU blue.

Custom Colors:

Different colors: red highlight, green highlight, yellow highlight, and blue highlight.

Math Formula Highlighting:

Inline math: $E = mc^2$ and $a^2 + b^2 = c^2$

Display math:

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

SMU
SINGAPORE MANAGEMENT UNIVERSITY

02 Components Design

Block Showcase

Standard

This is a standard block with SMU styling.

Alert

Warning block for important information.

Examples

Example block for demonstrating concepts.

Light box for highlighting supplementary information.

Tag

Tagged block for categorized content.

Theorem: Pythagorean Theorem

For a right triangle with sides a and b, and hypotenuse c:

$$a^2 + b^2 = c^2$$

Two Column Layout - Custom Width

The `\begin{twocolumns}` command defaults to 50% width for each column. You can use `\leftcol[0.6]` and `\rightcol[0.35]` to customize the column widths.

Wider Left Column (60%)

This column takes 60% of the available width, leaving 35% for the right column (plus some spacing).

Useful for:

- Main content vs. sidebar
- Text vs. small figure
- Detailed explanation vs. summary

Narrow Right (35%)

Smaller column for supplementary information.

Three Column Layout

Step 1
Initialize

Input: Data

Step 2
Process

Transform: Apply

Step 3
Output

Result: Done

Three columns are perfect for showing processes or comparisons.

Tables

Example: Performance Comparison

Method	Accuracy (%)	Time (ms)	Memory (MB)
Method A	95.3	12.5	128
Method B	97.1	18.2	256
Method C	96.8	15.7	192
Ours	**98.5**	**14.3**	**160**

Table: Performance comparison of different methods

Use `booktabs` package for professional-looking tables.

Mixed Lists in Columns

Unordered Lists (Itemize):

- First level item
- Another first level item
  - Second level nested item
  - Another second level item
    - Third level deeply nested
    - More third level content
  - Back to second level
- Back to first level
- Final first level item

Ordered Lists (Enumerate):

1. First step in the process
2. Second step with details
   2.1 Sub-step 2.1
   2.2 Sub-step 2.2
      2.2.1 Sub-sub-step 2.2.1
      2.2.2 Sub-sub-step 2.2.2
3. Third step returning to main level
4. Fourth and final step

Description Lists

Custom Description Lists:

API Application Programming Interface for software communication

SDK Software Development Kit containing tools and libraries

IDE Integrated Development Environment for coding

CI/CD Continuous Integration and Continuous Deployment

Description lists are perfect for glossaries or term definitions.

SMU
SINGAPORE MANAGEMENT UNIVERSITY

03 Code Rendering

Code Highlighting Features

The theme uses **minted** for professional code highlighting:

Features:

- Line numbers
- Syntax highlighting
- 49 color themes
- 100+ languages
- Inline code support

Supported Languages:

- Python, C, C++, Java
- JavaScript, TypeScript
- Go, Rust, Shell
- SQL, R, MATLAB
- And many more!

Important

Always use `[fragile]` option when frames contain code blocks!

Python: Fibonacci Sequence

```python
def fibonacci(n):
    """Calculate the nth Fibonacci number using recursion"""
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

def fibonacci_iterative(n):
    """Calculate Fibonacci iteratively (more efficient)"""
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

# Test the functions
for i in range(10):
    result = fibonacci_iterative(i)
    print("F(%d) = %d" % (i, result))
```

Inline Python: `lambda x: x**2`

C++: Template Function

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

template<typename T>
T findMax(const std::vector<T>& arr) {
    if (arr.empty()) {
        throw std::runtime_error("Empty vector");
    }
    return *std::max_element(arr.begin(), arr.end());
}

int main() {
    std::vector<int> numbers = {3, 7, 2, 9, 1, 5};
    std::cout << "Maximum: " << findMax(numbers) << std::endl;

    std::vector<double> values = {3.14, 2.71, 1.41};
    std::cout << "Maximum: " << findMax(values) << std::endl;

    return 0;
}
```

SQL and Shell Scripts

SQL Query

```sql
SELECT
    s.student_name,
    AVG(g.score) as avg_score,
    COUNT(*) as num_courses
FROM students s
JOIN grades g
    ON s.id = g.student_id
WHERE g.score >= 80
GROUP BY s.id, s.student_name
HAVING COUNT(*) >= 3
ORDER BY avg_score DESC
LIMIT 10;
```

Bash Script

```bash
#!/bin/bash

BACKUP_DIR="/backup"
DATE=$(date +%Y%m%d)
SOURCE="/data"

# Create backup
echo "Starting backup..."
tar -czf \
  "$BACKUP_DIR/bk_$DATE.tar.gz" \
  "$SOURCE"

if [ $? -eq 0 ]; then
    echo "Backup completed"
else
    echo "Backup failed!"
    exit 1
fi
```

Inline Code Examples

Inline code for different languages:

- Python: `def hello(): return "world"`
- C: `int *ptr = malloc(sizeof(int));`
- Java: `List<String> names = new ArrayList<>();`
- JavaScript: `const arr = [1, 2, 3].map(x => x * 2);`

Generic Inline Code

You can also use `\code[language]{code}` for any language:

- Generic: Use `printf("Hello")` for output
- Commands: Run `git commit -m "message"`
- Paths: See file `/etc/config.conf`

Theme Components Summary

Text & Colors:

- SMU brand colors
- Custom color commands
- Text highlighting
- Bold color variants
- Math highlighting

Blocks:

- Standard blocks
- Alert blocks
- Example blocks
- Light boxes
- Tagged blocks
- Theorem boxes

Code:

- Minted integration
- 49 color themes
- 100+ languages
- Line numbers
- Inline code

Layout:

- 2-column layouts
- 3-column layouts
- Custom widths
- Progress bar
- Section pages

Quick Reference

Getting Started

Load the theme: `\usepackage{smu}`

Code Highlighting

For code blocks: `\begin{frame}[fragile]`
Use environments: `pycode`, `cppcode`, `jscode`, etc.

Columns

Two columns: `\begin{twocolumns}...\end{twocolumns}`
Three columns: `\begin{threecolumns}...\end{threecolumns}`

Documentation

See `README.md` for detailed documentation and examples.

Customization

The theme is highly customizable:

Colors	Edit `smucore/smucolors.def`
Fonts	Edit `smucore/smufonts.def`
Code Style	Edit `smucore/smucode.def`
Commands	Edit `smucore/smucommands.def`
Blocks	Edit `smucore/smublocks.def`
Templates	Edit `smucore/smutemplates.def`

All components are modular and can be customized independently!

Resources

Documentation

- `README.md` - Main documentation
- `main_test.tex` - This demo file
- Source files in `smucore/` directory

External Resources

- Minted: `https://ctan.org/pkg/minted`
- Beamer: `https://ctan.org/pkg/beamer`
- Pygments Styles: `https://pygments.org/styles/`

Reference Test

MtA-based Approaches (Multiplicative-to-Additive)

- Paillier Encryption: Requires strong-RSA assumption.
  [GGN+16; Lin+17; GG+18; CGG+20; XLX+24]
- CL Encryption: Based on Class Groups.
  [CCL+19; CCL+20; YCX+21; DMZ+21]
- Oblivious Transfer: No extra assumptions and computationally efficient.
  [DKL+18; DKL+19; DKL+24]
- Joye-Libert: Based on revisited Joye-Libert encryption.
  [XAL+23]

Threshold CL Encryption

- Decryption key of CL shared among n parties.
  [BDO+23; WMY+23; WMC+24; TX+25; JTX+25]

Others

- Pseudorandom Correlation Generator [ANO+22], ...

References

[ANO+22] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits, Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In: *2022 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2022, pp. 2554–2572.

[BDO+23] Lennart Braun, Ivan Damgård, and Claudio Orlandi, Secure multiparty computation from threshold encryption based on class groups. In: *Annual International Cryptology Conference*, Springer, 2023, pp. 613–645.

[CCL+19] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker, Two-party ECDSA from hash proof systems and efficient instantiations. In: *Annual International Cryptology Conference*, Springer, 2019, pp. 191–221.

[CCL+20] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker, Bandwidth-efficient threshold EC-DSA. In: *IACR International Conference on Public-Key Cryptography*, Springer, 2020, pp. 266–296.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled, UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1769–1787.

[DKL+18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat, Secure two-party threshold ECDSA from ECDSA assumptions. In: *2018 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2018, pp. 980–997.

[DKL+19] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat, Threshold ECDSA from ECDSA assumptions: The multiparty case. In: *2019 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2019, pp. 1051–1066.

[DKL+24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat, Threshold ECDSA in three rounds. In: *2024 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2024, pp. 3053–3071.

[DMZ+21] Yi Deng, Shunli Ma, Xinxuan Zhang, Hailong Wang, Xuyang Song, and Xiang Xie, Promise Σ-protocol: How to construct efficient threshold ECDSA from encryptions based on class groups. In: *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, 2021, pp. 557–586.

[GG+18] Rosario Gennaro and Steven Goldfeder, Fast multiparty threshold ECDSA with fast trustless setup. In: *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1179–1194.

[GGN+16] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan, Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In: *International conference on applied cryptography and network security*, Springer, 2016, pp. 156–174.

[JTX+25] Bowen Jiang, Guofeng Tang, and Haiyang Xue, Three-Round (Robust) Threshold ECDSA from Threshold CL Encryption. In: *Australasian Conference on Information Security and Privacy*, Springer, 2025, pp. 224–244.

[Lin+17] Yehuda Lindell, Fast secure two-party ECDSA signing. In: *Journal of Cryptology* 34.4 (2017), p. 44.

[TX+25] Guofeng Tang and Haiyang Xue, Robust Threshold ECDSA with Online-Friendly Design in Three Rounds. In: *2025 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2025, pp. 203–221.

[WMC+24] Harry WH Wong, Jack PK Ma, and Sherman SM Chow, Secure multiparty computation of threshold signatures made more efficient. In: *NDSS*, The Internet Society, 2024.

[WMY+23] Harry WH Wong, Jack PK Ma, Hoover HF Yin, and Sherman SM Chow, Real Threshold ECDSA. In: *NDSS*, 2023.

[XAL+23] Haiyang Xue, Man Ho Au, Mengling Liu, Kwan Yin Chan, Handong Cui, Xiang Xie, Tsz Hon Yuen, and Chengru Zhang, Efficient multiplicative-to-additive function from Joye-Libert cryptosystem and its application to threshold ECDSA. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2974–2988.

[XLX+24] Zhikang Xie, Mengling Liu, Haiyang Xue, Man Ho Au, Robert H Deng, and Siu-Ming Yiu, Direct Range Proofs for Paillier Cryptosystem and Their Applications. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 899–913.

[YCX+21] Tsz Hon Yuen, Handong Cui, and Xiang Xie, Compact zero-knowledge proofs for threshold ECDSA with trustless setup. In: *IACR International Conference on Public-Key Cryptography*, Springer, 2021, pp. 481–511.

SMU
SINGAPORE MANAGEMENT UNIVERSITY

Thank You

Q & A